

Chapter-3: Word Vector Representations

Dr. Shiv Shakti Shrivastava
Professor
Department of CSE

Content

- 1. Word Vector Representations**
- 2. Language Modeling**
- 3. What is Word Embedding?**
- 4. Word2Vec: Skip-Gram Model**
- 5. Word vector representations**
- 6. Implementations of Word Embeddings**
- 7. Sparse Matrix**
- 8. Continuous Bowl of Words(CBOW)**

Unit-3

Syllabus - Word Vector Representations

- Introduction to Word Embedding: Neural Language Model, Word2Vec: Skip-Gram Model, Model Components: Architecture, Hidden Layer, Output Layer, CBOW Model, Subsampling Frequent Words: Negative Sampling. Efficient Softmax approximations, Sampling-based approximations for Softmax, Glove, Cross-lingual word embedding models

Word Vector Representations

- It is an approach for representing words and documents.
- Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space.
- It allows words with similar meaning to have a similar representation.
- They can also approximate meaning.
- A word vector with 50 values can represent 50 unique features.

Language Modeling

- **Language Modeling** is the task of predicting the next word or character in a document.
- This technique can be used to train language models that can further be applied to a wide range of natural language tasks like text generation, text classification and question answering.

- The common types of language modeling techniques involve:
 1. N-gram Language Models
 2. Unigram
 3. Bidirectional
 4. Exponential
 5. Continuous space
 6. Neural Language Models

How language modeling works

- Language models determine word probability by analyzing text data.
- They interpret this data by feeding it through an algorithm that establishes rules for context in natural language.
- Then, the model applies these rules in language tasks to accurately predict or produce new sentences.
- The model essentially learns the features and characteristics of basic language and uses those features to understand new phrases.

Uses and examples of language modeling

Language models are the backbone of natural language processing (NLP). Below are some NLP tasks that use language modeling, what they mean, and some applications of those tasks:

1. Speech recognition -- involves a machine being able to process speech audio. This is commonly used by voice assistants like Siri and Alexa.
2. Machine translation -- involves the translation of one language to another by a machine.
 - Google Translate and Microsoft Translator are two programs that do this.

3. Parts-of-speech tagging -- involves the markup and categorization of words by certain grammatical characteristics.
 - This is utilized in the study of linguistics, first and perhaps most famously in the study of the Brown Corpus, a body of composed of random English prose that was designed to be studied by computers.
 - This corpus has been used to train several important language models, including one used by Google to improve search quality.
4. Parsing -- involves analysis of any string of data or sentence that conforms to formal grammar and syntax rules. In language modeling, this may take the form of sentence diagrams that depict each word's relationship to the others. Spell checking applications use language modeling and parsing.

5. Sentiment analysis -- involves determining the sentiment behind a given phrase.

Specifically, it can be used to understand opinions and attitudes expressed in a text.

- Businesses can use this to analyze product reviews or general posts about their product, as well as analyze internal data like employee surveys and customer support chats.

6. Optical Character Recognition (OCR) -- involves the use of a machine to convert images of text into machine encoded text.

- The image may be a scanned document or document photo, or a photo with text somewhere in it -- on a sign,
- It is often used in data entry when processing old paper records that need to be digitized.
- It can also be used to analyze and identify handwriting samples.

7. **Information retrieval** -- involves searching in a document for information, searching for documents in general, and searching for metadata that corresponds to a document. Web browsers are the most common information retrieval applications.

Features: Anything that relates words to one another.

Eg: Age, Sports, Fitness, Employed etc. Each word vector has values corresponding to these features.

Goal of Word Embeddings:

- To reduce dimensionality
- To use a word to predict the words around it
- Inter word semantics must be captured

What is Word Embedding?

- **Word Embedding** is a word representation type that allows machine learning algorithms to understand words with similar meanings.
- It is a language modeling and feature learning technique to map words into vectors of real numbers using neural networks, probabilistic models, or dimension reduction on the word co-occurrence matrix.
- Some word embedding models are Word2vec (Google), Glove (Stanford), and fastest (Facebook).

Word Embedding is also called as -

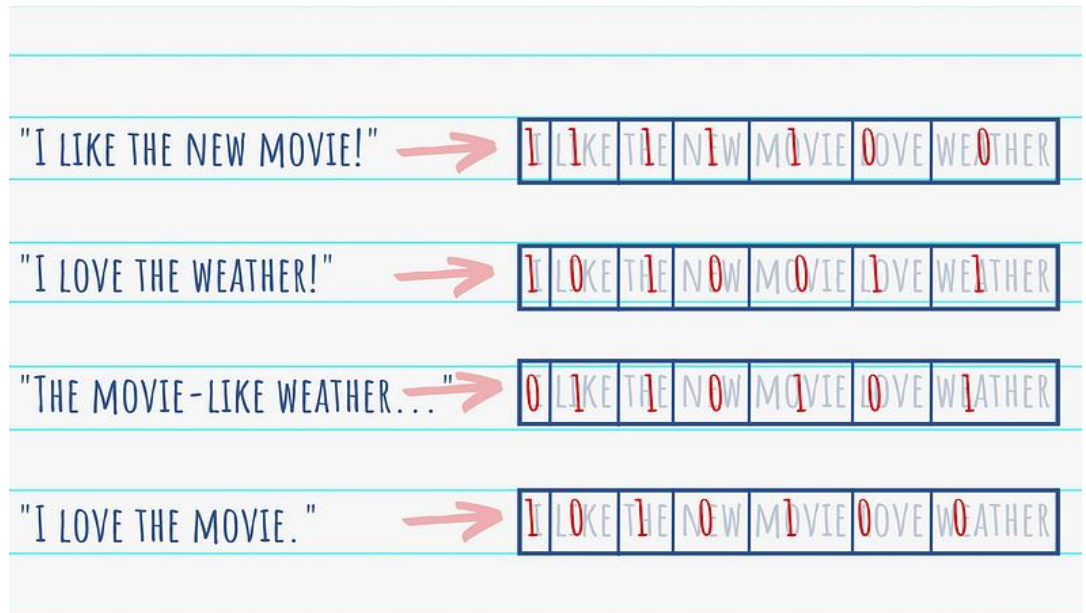
- Word Embedding is also called as
- Distributed Semantic Model or
- Distributed Represented or
- Semantic Vector Space or
- Vector Space Model.
- As you read these names, you come across the word semantic which means categorizing similar words together.

Word embeddings

- A preliminary method is a “bag of words” model which encodes words using a one-hot scheme.
- If our dataset contains the sentences:
 - “I like the new movie!”,
 - “I love the weather.”

- Then we can have a vector representation of the words as:

- I [1,0,0,0,0,0,0]
- like [0,1,0,0,0,0,0]
- the [0,0,1,0,0,0,0]
- new [0,0,0,1,0,0,0]
- movie [0,0,0,0,1,0,0]
- love [0,0,0,0,0,1,0]
- weather [0,0,0,0,0,0,1]



- The sentences will then be represented as:
- [1,1,1,1,1,0,0] and [1,0,1,0,0,1,1]

Word2Vec

- Word2Vec creates a representation of each word present in our vocabulary into a vector.
- Words used in similar contexts or having semantic relationships are captured effectively through their closeness in the vector space- effectively speaking similar words will have similar word vectors.
- Let us consider a classic example:
- “king”, “queen”, “man”, “girl”, “prince”

		KING	QUEEN	MAN	GIRL	PRINCE
Royalty		0.96	0.98	0.05	0.56	0.95
Masculinity		0.92	0.07	0.90	0.09	0.85
Femininity		0.08	0.93	0.10	0.91	0.15
Age		0.67	0.71	0.56	0.11	0.42

What we then observe is:

- As expected, “king”, “queen”, “prince” have similar scores for “royalty” and “girl”, “queen” have similar scores for “femininity”.
- An operation that removes “man” from “king”, would yield in a vector very close to “queen” (“king”- “man” +”Woman”= “queen”)
- Vectors “king” and “prince” have the same characteristics, except for age, telling us how they might possibly be semantically related to each other.

Where is Word Embedding used?

- **Word embedding** helps in feature generation, document clustering, text classification, and natural language processing tasks. Let us list them and have some discussion on each of these applications.
- **Compute Similar Words:** Word embedding is used to suggest similar words to the word being subjected to the prediction model. Along with that it also suggests dissimilar words, as well as most common words.

Where is Word Embedding used?

- **Create a group of related words:** It is used for semantic grouping which will group things of similar characteristic together and dissimilar far away.
- **Feature for text classification:** Text is mapped into arrays of vectors which is fed to the model for training as well as prediction. Text-based classifier models cannot be trained on the string, so this will convert the text into machine trainable form. Further its features of building semantic help in text-based classification.

Where is Word Embedding used?

- **Document clustering:** is another application where Word Embedding Word2vec is widely used.
- **Natural language processing:** There are many applications where word embedding is useful and wins over feature extraction phases such as
 - parts of speech tagging,
 - sentimental analysis, and
 - syntactic analysis.

Why Word Embeddings?

- Traditional methods like Bag-of-Words (BoW) lack semantic understanding.
- Word embeddings overcome this limitation by representing words as dense vectors.
- These vectors capture relationships between words based on their context.
- **How Word Embeddings Work?**
- Neural language models learn word embeddings as part of their training process.
- Words are represented as distributed representations, where each dimension contributes to the meaning of the word.
- Similar words have similar vector representations.

Applications of Word Embeddings

- Text classification
- Sentiment analysis
- Machine translation
- Information retrieval
- Recommendation systems

Word Embedding: Neural Language Model

- A word embedding is a numerical representation of a word in a lower-dimensional space.
- Word embeddings capture the semantic and syntactic information of words.
- They are used in a variety of natural language processing tasks, such as text classification, sentiment analysis, and machine translation.
- Word embeddings are learned by neural language models, such as Word2Vec and GloVe.
- Word embeddings are a powerful tool for representing words in a way that is more meaningful than traditional methods, such as bag-of-words.

Word2Vec: Skip-Gram Model

- Word2Vec is a neural network model that learns word embeddings.
- It is based on the idea that the meaning of a word can be inferred from its context.
- Word2Vec has two main architectures: the skip-gram model and the continuous bag-of-words (CBOW) model.
- The skip-gram model predicts the surrounding words of a target word.
- The CBOW model predicts the target word from its surrounding words.
- Word2Vec is a powerful tool for learning word embeddings that capture the semantic and syntactic information of words.

Skip-Gram Model

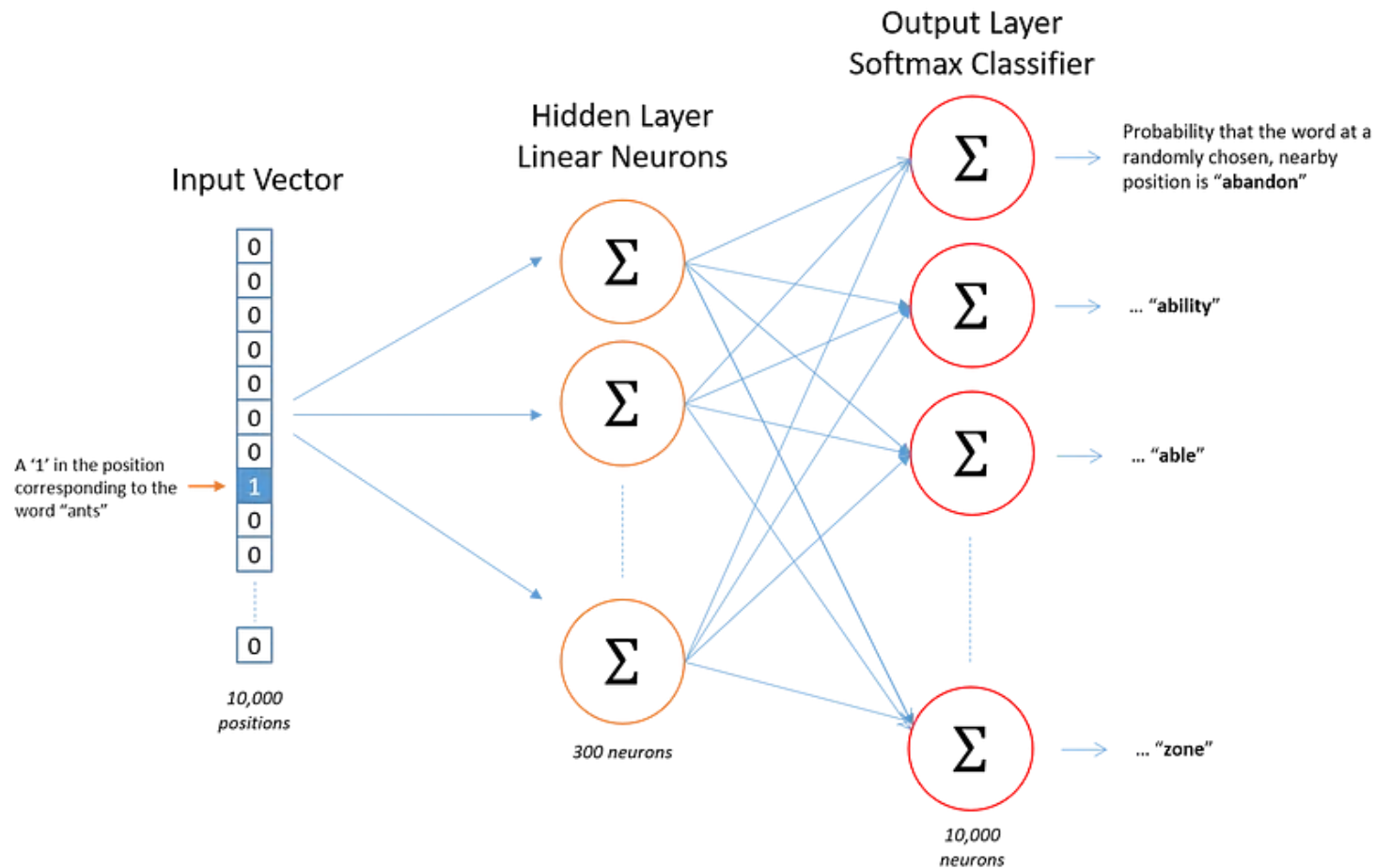
- Predicts surrounding context words given a target word.
- Takes a target word as input and tries to predict the surrounding words within a defined window.
- Learns to represent words in a way that maximizes the probability of predicting correct context words.

Architecture of Skip-Gram Model

- Input layer: Represents the target word.
- Projection layer: Maps the input word to a lower-dimensional space.
- Output layer: Predicts the probability of each word in the vocabulary being a context word.

Training Process

- Iterates over a corpus of text.
- For each word, creates training examples by considering surrounding words within a window.
- Updates model parameters to maximize the likelihood of predicting correct context words.
- **Applications of Skip-Gram**
- Similar to general word embeddings applications.
- Effective for capturing semantic and syntactic relationships between words.



Overview of Word2Vec

- Word2vec is a combination of models used to represent distributed representations of words in a corpus C. Word2Vec (W2V) is an algorithm that accepts text corpus as an input and outputs a vector representation for each word, as shown in the diagram below:



There are two flavors of this algorithm namely: **CBOW** and **Skip-Gram**.

Given a set of sentences (also called corpus) the model loops on the words of each sentence and either tries to use the current word w in order to predict its neighbors (i.e., its context), this approach is called "Skip-Gram", or it uses each of these contexts to predict the current word w , in that case the method is called "Continuous Bag Of Words" (CBOW).

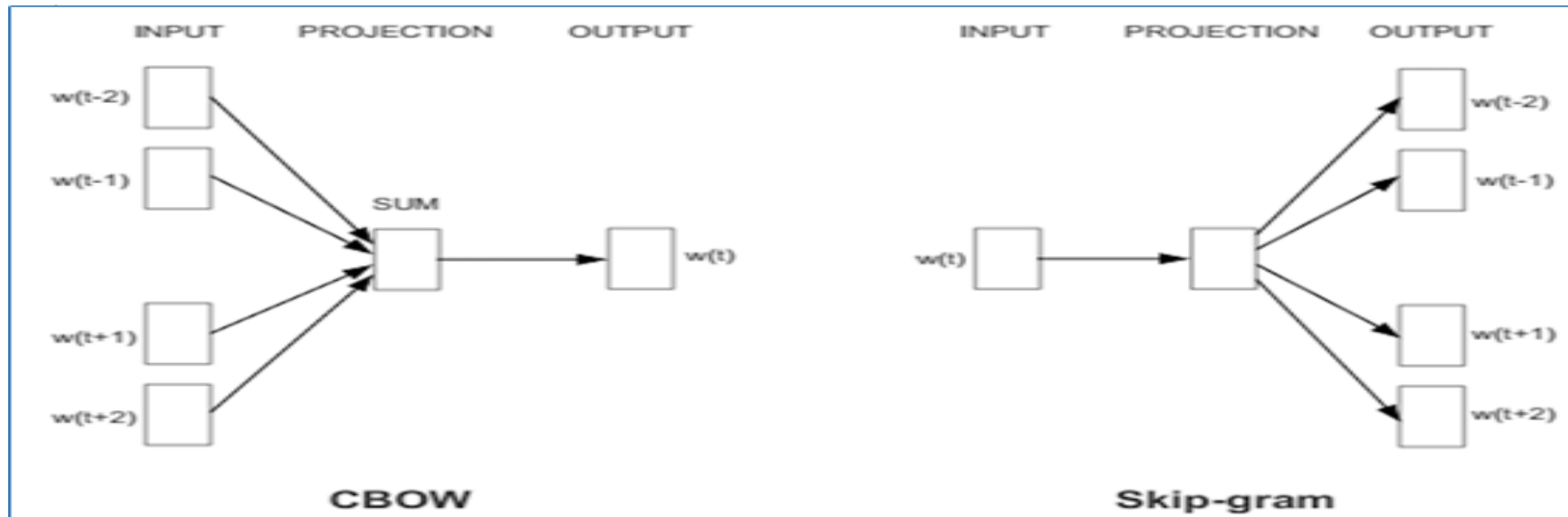
To limit the number of words in each context, a parameter called "window size" is used.

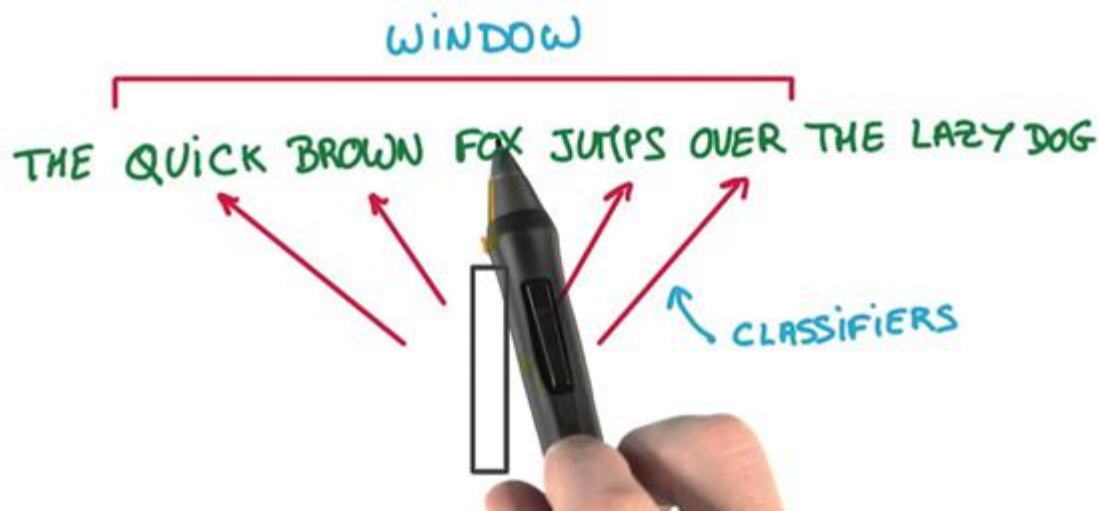
Source Text

Training Samples

The quick brown fox jumps over the lazy dog. →	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog. →	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog. →	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog. →	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

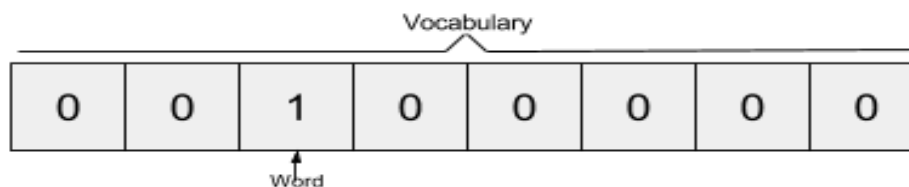
- It does so in one of two ways, either using context to predict a target word (a method known as continuous bag of words, or CBOW) or using a word to predict a target context, which is called skip-gram.
- We use the latter method because it produces more accurate results on large datasets.





- we will have a vector with all zeros and a 1 which represents the corresponding word in the vocabulary.
- This encoding technique is called one-hot encoding. Considering our example,
- if we have a vocabulary made out of the words “the”, “quick”, “brown”, “fox”, “jumps”, “over”, “the” “lazy”, “dog”, the word “brown” is represented by this vector: [0, 0, 1, 0, 0, 0 ,0 ,0 ,0].

- The Skip-gram model takes in a corpus of text and creates a hot-vector for each word.
- A hot vector is a vector representation of a word where the vector is the size of the vocabulary (total unique words).
- All dimensions are set to 0 except the dimension representing the word that is used as an input at that point in time. Here is an example of a hot vector:



- The above input is given to a neural network with a single hidden layer.

There are two ways in which we can develop these embeddings:

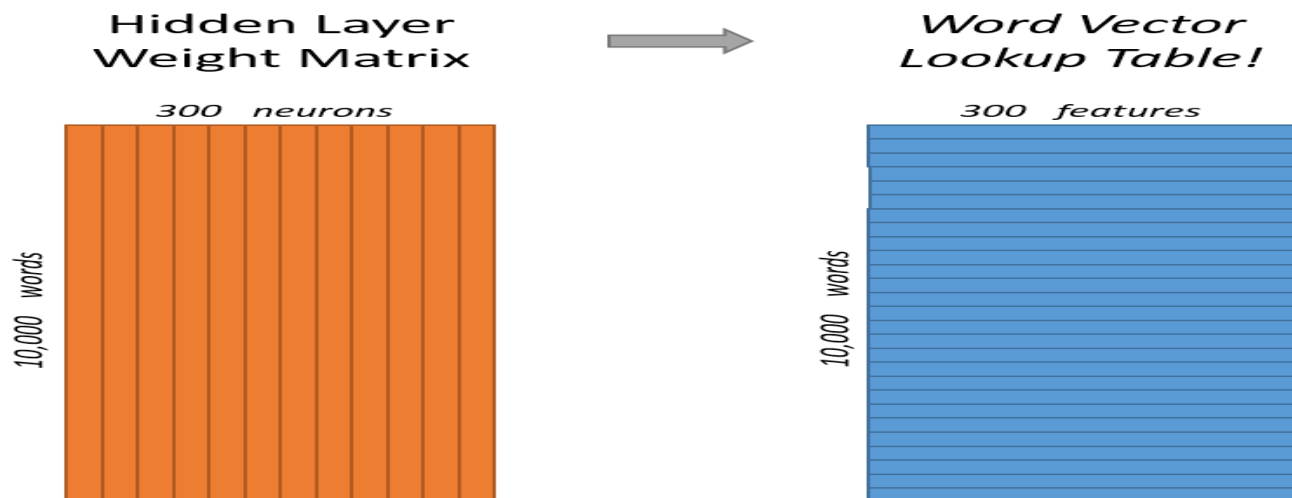
1. Continuous Bag-Of-Words (CBOW)

CBOW predicts the target-word based on its surrounding words.

For example, consider the sentence, “The cake was chocolate flavored”.

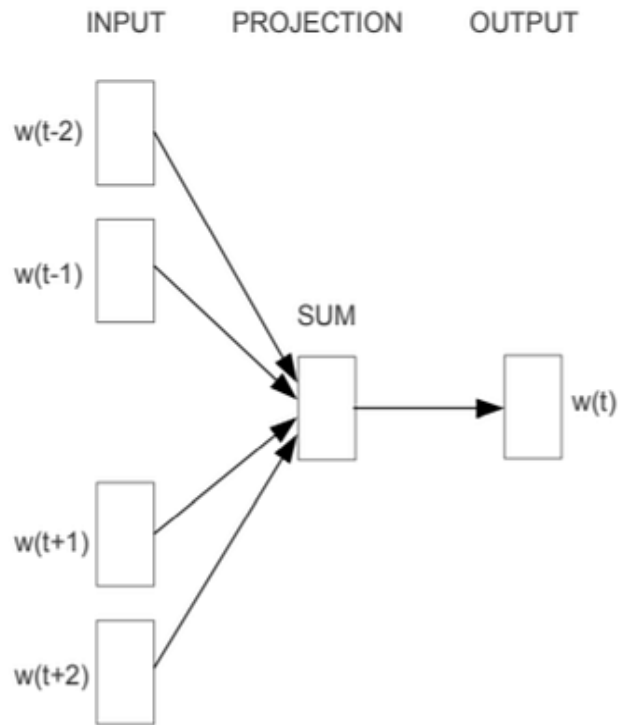
The model will then iterate over this sentence for different target words, such as:

“The ____ was chocolate flavored” being inputs and “cake” being the target word.

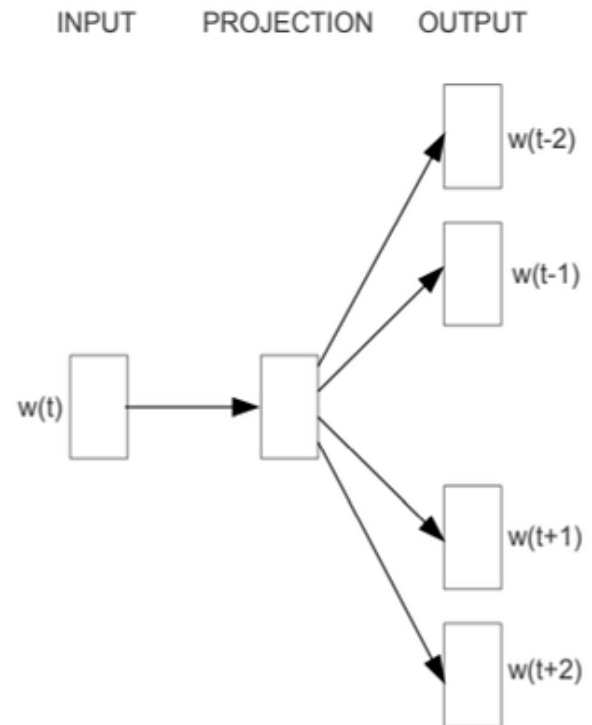


- **Skipgram** works in the exact opposite way to CBOW. Here, we take an input word and expect the model to tell us what words it is expected to be surrounded by.
- Taking the same example, with “cake” we would expect the model to give us “The”, “was”, “chocolate”, “flavored” for the given instance.
- The statistical interpretation of this is that we treat each context-target pair as a new observation. Skipgrams work well with small datasets and can better represent less frequent words.

Word Embedding



CBOW



Skip-gram

Word Embedding

WORD EMBEDDING (Feature Representation)

Analogy {Gender} (2000) (5000) (6000) (4000) (1000) (7000)

	Boy	GIRL	KING	QUEEN	APPLE	MANGO
Boy → GIRL	-1	1	-0.92	0.93	0.0	0.1
$x_1 - x_2 = \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	0.01	0.02	0.95	0.96	-0.02	0.01
Cosine Similarity	0.03	0.02	0.7	0.6	0.95	0.92
KING = $\begin{bmatrix} x-2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	⋮	⋮	⋮	⋮	⋮	⋮

Low Dimension & Dense Matrix

Word vector representations

- Word vector representations are dense and continuous, as opposed to sparse and discrete representations like one-hot encoding or bag-of-words.
- Each dimension of the vector represents a different aspect of the word's meaning, allowing for more nuanced and contextualized representations.
- The main benefit of word vector representations is their ability to capture the meaning and relationships between words.
- Words with similar meanings or that often appear in similar contexts will have similar vector representations, making it possible to measure the similarity between words mathematically.
- This enables machines to understand and work with textual data more effectively.

- For example, consider the word vectors for "king" and "queen." In a well-trained word embedding model, these two words will have vector representations that are close to each other in the vector space.
- Similarly, words like "man" and "woman" will have similar vector representations. This geometric proximity in the vector space reflects the semantic relationships between these words.

- Word vector representations have several useful properties.
- First, they can capture semantic similarities, allowing for tasks like word similarity measurement, analogy completion (e.g., "king" is to "queen" as "man" is to what?), and word sense disambiguation.
- Second, they can capture syntactic regularities, such as verb tense or pluralization.

For example, the vector representation of "walked" might be close to the vector representation of "ran" or "jogged."

How are Word Embeddings used?

- They are used as input to machine learning models.
Take the words —-> Give their numeric representation
- —-> Use in training or inference
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

Implementations of Word Embeddings

- Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data.
- They try to preserve syntactical and semantic information.
- The methods such as Bag of Words(BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information.

Sparse Matrix

- A [matrix](#) is a two-dimensional data object made of m rows and n columns, therefore having total $m \times n$ values.
- If most of the elements of the matrix have **0 value**, then it is called a sparse matrix.

Why to use Sparse Matrix instead of simple matrix ?

- **Storage:** There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
- **Computing time:** Computing time can be saved by logically designing a data structure traversing only non-zero elements..

TF-IDF

- TF-IDF (term frequency-inverse document frequency) is a statistical measure that evaluates how relevant a word is to a document in a collection of documents.
- This is done by multiplying two metrics: how many times a word appears in a document and the inverse document frequency of the word across a set of documents.

Word Embeddings give a solution to these problems.

- In these algorithms, the size of the vector is the number of elements in the vocabulary.
- We can get a sparse matrix if most of the elements are zero.
- Large input vectors will mean a huge number of weights which will result in high computation required for training.

- Let's take an example to understand how word vector is generated by taking emoticons which are most frequently used in certain conditions and transform each emoji into a vector and the conditions will be our features.

Happy			
Sad			
Excited			
Sick			

The emoji vectors for the emojis will be:

[happy, sad, excited, sick]

happy = [1,0,1,0]

sad = [0,1,0,1]

excited = [0,0,1,1].....etc.

- In a similar way, we can create word vectors for different words as well on the basis of given features.
- The words with similar vectors are most likely to have the same meaning or are used to convey the same sentiment.

Two different approaches to get Word Embeddings:

Word2Vec:

- In Word2Vec every word is assigned a vector.
- We start with either a random vector or **one-hot vector**.

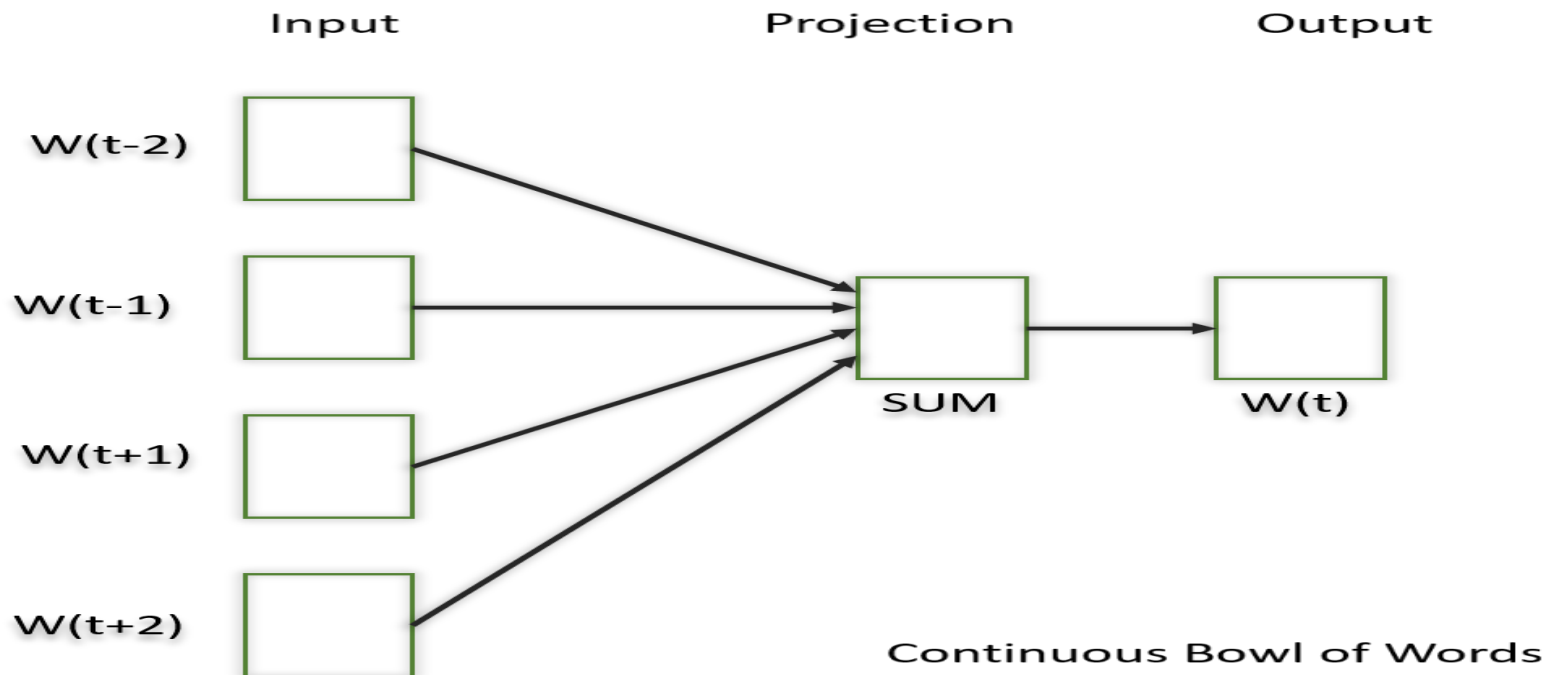
One-Hot vector:

- A representation where only one bit in a vector is 1.
- If there are 500 words in the corpus then the vector length will be 500.
- After assigning vectors to each word we take a window size and iterate through the entire corpus. While we do this there are two **neural embedding methods** which are used:

- The working mechanism of CBOW can be mathematically represented as follows:
 1. Let $x(1), x(2), \dots, x(n)$ be the context words, where $x(i)$ is a one-hot encoded vector.
 2. Let $W(1)$ be the weight matrix connecting the input layer to the hidden layer, and $W(2)$ be the weight matrix connecting the hidden layer to the output layer.
 3. Let h be the hidden layer, which is the average of the input vectors,
$$h = 1/n * (x(1) + x(2) + \dots + x(n))$$
 4. Let y be the output layer, which is the probability distribution over the vocabulary, $y = \text{softmax}(W(2) * h)$
 5. The target word is selected as the word with the highest probability in y .

1.1) Continuous Bowl of Words(CBOW)-

- In this model what we do is we try to fit the neighboring words in the window to the central word.



Word2Vec - CBOW

Window Size = 2 Target word context words

Mangoes are **sweet** and yellow

Training samples #1

Context words

Mangoes, are
, and, yellow

Target Word

Sweet

Mangoes are sweet **and** yellow

Training samples #2

Context words

are, Sweet, yellow

Target Word

and

Center Word	Context Words
sweet	Mangoes, are, And, yellow
And	Are, sweet, yellow



Word2Vec - CBOW

Window Size = 2 Target word context words

Mangoes are **sweet** and yellow

Training samples #1

Context words

Mangoes, are
, and, yellow

Target Word

Sweet

Mangoes are sweet **and** yellow

Training samples #2

Context words

are, Sweet, yellow, we and

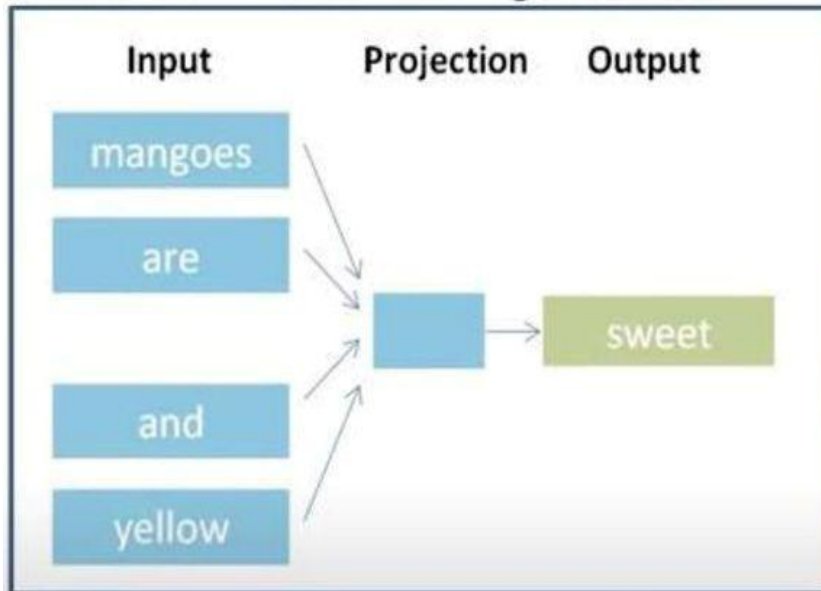
Target Word

Center Word	Context Words
$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$ $\begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$ $\begin{bmatrix} 1 & 0 & 0 & 1 \end{bmatrix}$ $\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$
$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$ $\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$ $\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$

Word2Vec

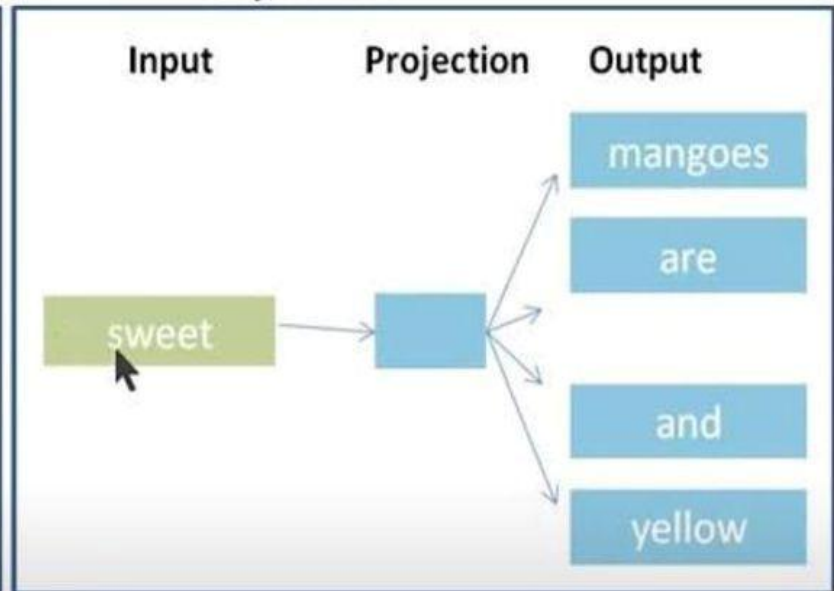


•CBOW – Continuous Bag of Words



Neighbor words are input and output is the center word.
Works good with large datasets.
Better representation for frequent words than rarer.

•Skip Gram



Input is centre word and output is context words (neighbour words).
Works well with small datasets.
Skip-gram identifies rarer words better.



Word2Vec - CBOW

Window Size = 2 Target word context words

Mangoes are **sweet** and yellow, We will buy

Training samples #1

Context words

Target Word

Mangoes, are
, Sweet, and, yellow **Sweet**

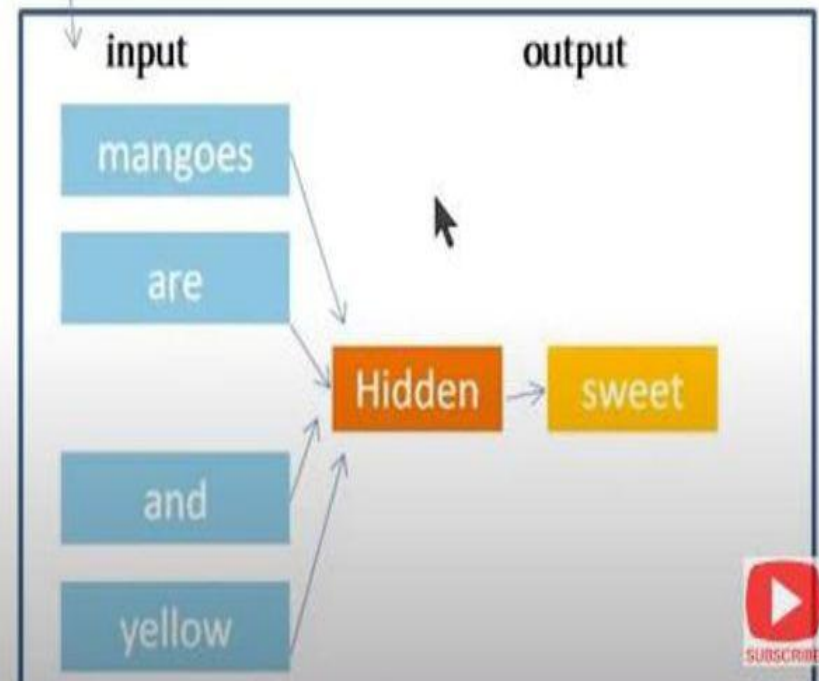
Mangoes are sweet **and** yellow, We will buy

Training samples #2

Context words

Target Word

are, Sweet, yellow, we **and**



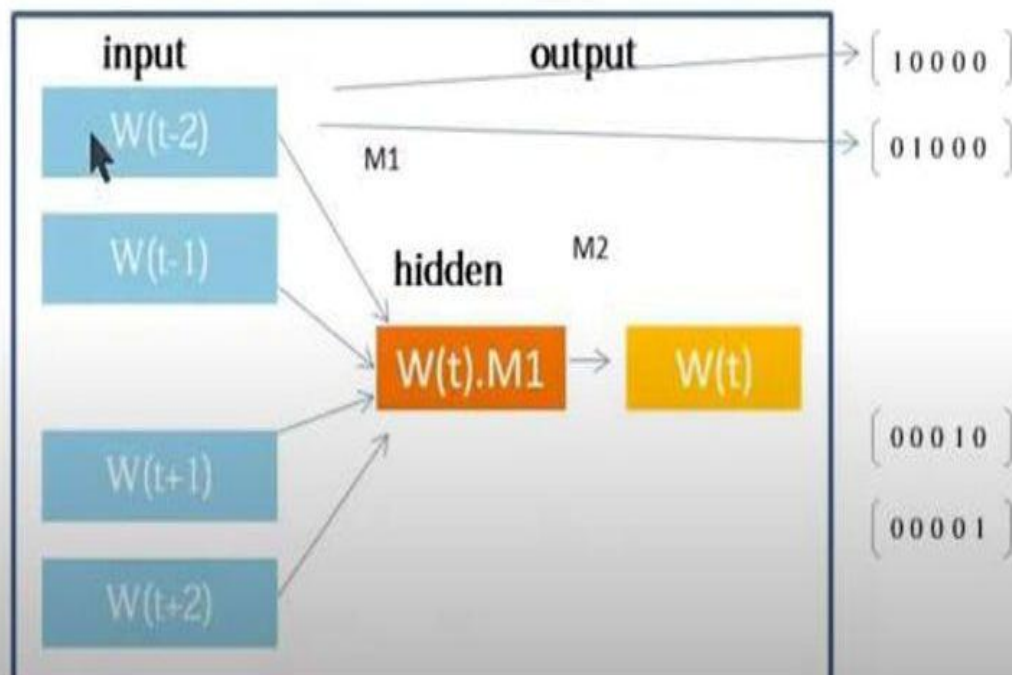
Word2Vec - CBOW model

M1 & M2 -> weight matrix

No activation function at hidden layer

Input

Input-hidden



$$M1(\text{weight}) = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 2 \\ 3 & 4 & 2 \\ 2 & 3 & 4 \\ 3 & 1 & 1 \end{bmatrix}$$



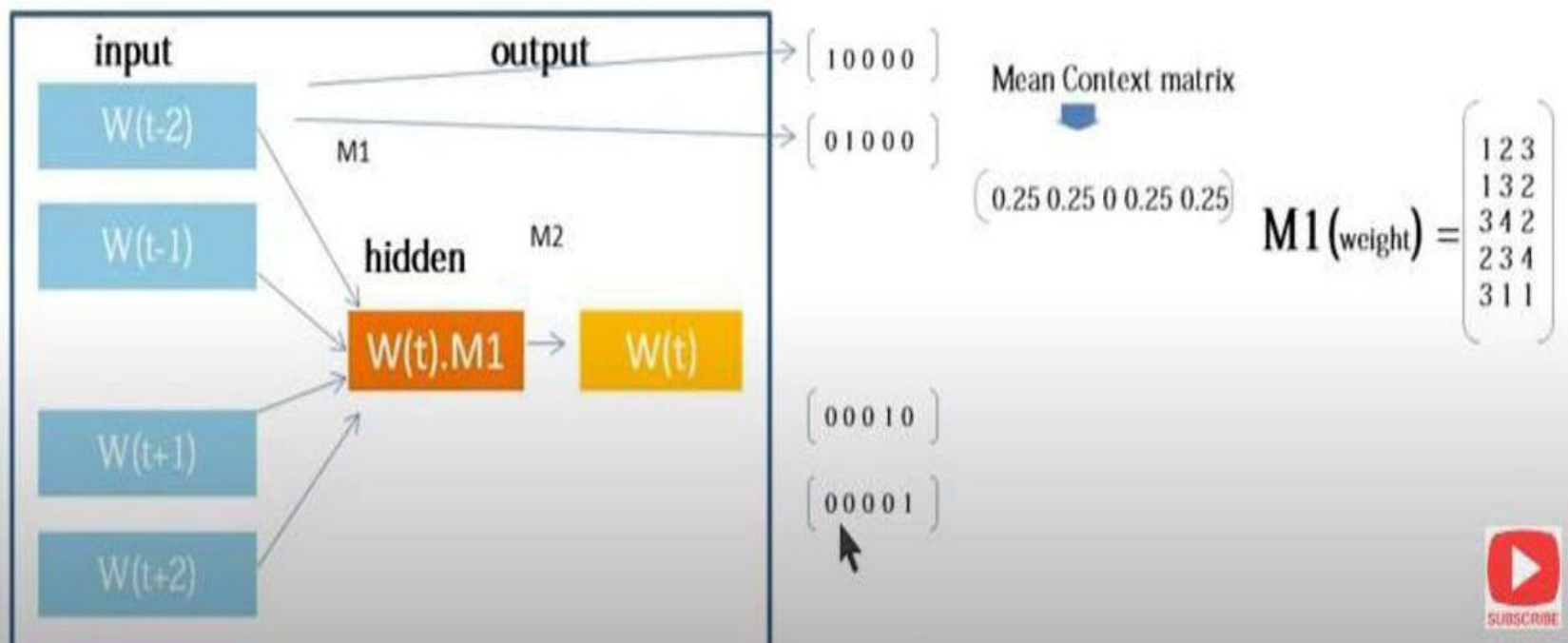
Word2Vec - CBOW model

M1 & M2 -> weight matrix

No activation function at hidden layer

Input-hidden

Input



Word2Vec - CBOW model

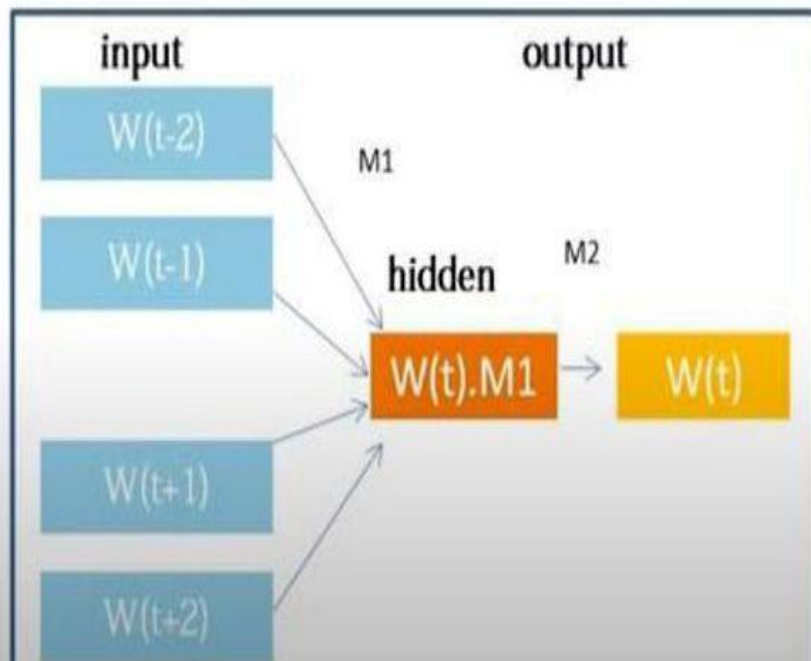
M1 & M2 -> weight matrix

No activation function at hidden layer

Input

Input-hidden

$$H = M1^T X$$



$$\begin{bmatrix} 1 & 1 & 3 & 2 & 3 \\ 2 & 3 & 4 & 3 & 1 \\ 3 & 2 & 2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 0.25 & 0.25 & 0 & 0.25 & 0.25 \end{bmatrix} = \begin{bmatrix} 1.7 \\ 2.7 \\ 2.5 \end{bmatrix}$$

Hidden O/p



Word2Vec – CBOW model

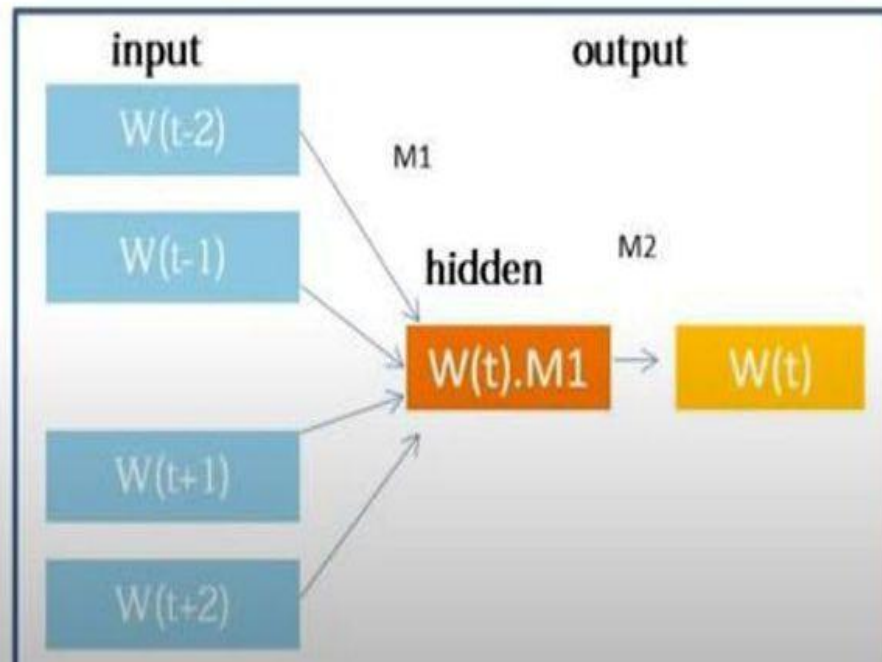
M1 & M2 - > weight matrix

No activation function at hidden layer

output

Hidden-output

$$U = M2^T H$$



$$M2(\text{weight}) = \begin{pmatrix} 2 & 1 & 2 & 2 & 3 \\ 1 & 3 & 1 & 3 & 1 \\ 2 & 2 & 3 & 2 & 2 \end{pmatrix}$$

Hidden O/p

$$\begin{pmatrix} 1.7 \\ 2.7 \\ 2.5 \end{pmatrix}$$

Word2Vec - CBOW model

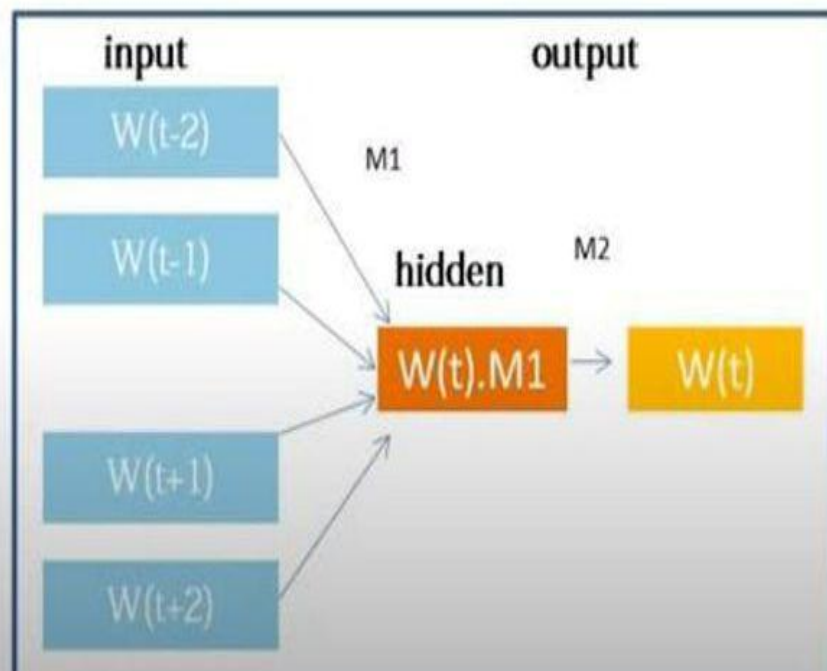
M1 & M2 -> weight matrix

No activation function at hidden layer

Hidden-output

output

$$U = M2^T H$$



$$M2^T \text{ Hidden O/p} = \begin{pmatrix} 2 & 1 & 2 \\ 1 & 3 & 2 \\ 2 & 1 & 3 \\ 2 & 3 & 2 \\ 3 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1.7 \\ 2.7 \\ 2.5 \end{pmatrix} = \begin{pmatrix} 2 \times 1.7 + 1 \times 2.7 + 2 \times 2.5 \\ 1 \times 1.7 + 3 \times 2.7 + 2 \times 2.5 \\ 2 \times 1.7 + 1 \times 2.7 + 3 \times 2.5 \\ 2 \times 1.7 + 3 \times 2.7 + 2 \times 2.5 \\ 3 \times 1.7 + 1 \times 2.7 + 2 \times 2.5 \end{pmatrix} = \begin{pmatrix} 9.11 \\ 14.8 \\ 13.6 \\ 19.0 \\ 15.3 \end{pmatrix}$$

O/p - U



Word2Vec - CBOW model

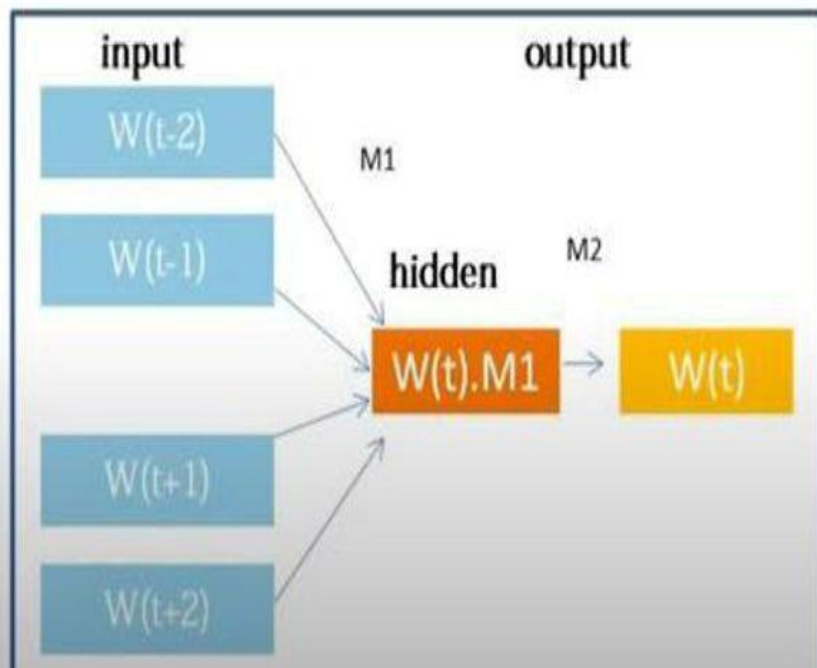
M1 & M2 -> weight matrix

No activation function at hidden layer

output

Hidden-output

$$U = M2^T H$$



U =

O/p

9.11
14.8
13.6
19.0
15.3



Softmax



Y

0.11
0.25
0.13
0.34
0.17



Softmax Activation Function

Softmax : Final output is a probability distribution.
distribution of probability at output layer.

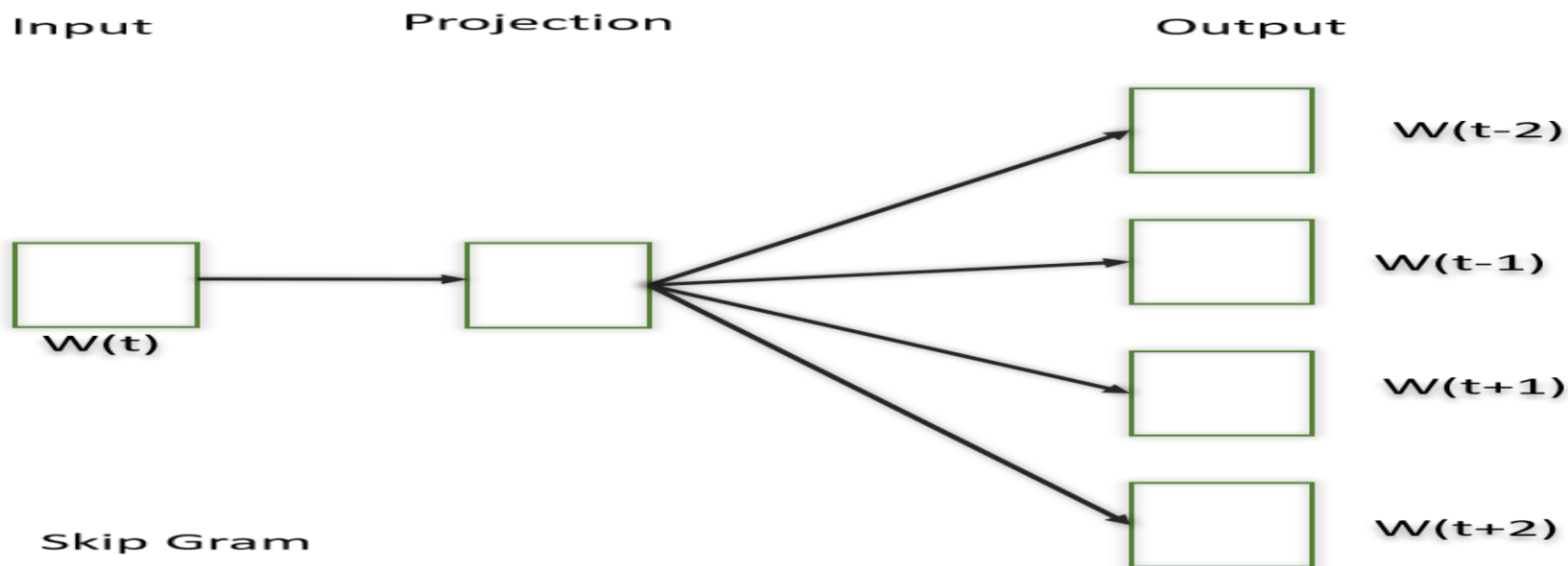
In normal scenario there are only two things , will happen or not like .7 or .3
but now here we have more than 2 cases.

Out put = [0.2 0.5 0.2 0.1]

$0.2 + 0.5 + 0.2 + 0.1 = 1$ all output values are between 0 to 1

Skip Gram-

- In this model, we try to make the central word closer to the neighboring words.
- It is the complete opposite of the CBOW model.
- It is shown that this method produces more meaningful embeddings.



- After applying the above neural embedding methods, we get trained vectors of each word after many iterations through the corpus.
- These trained vectors preserve syntactical or semantic information and are converted to lower dimensions.
- The vectors with similar meaning or semantic information are placed close to each other in space.

GloVe:

- This is another method for creating word embeddings. In this method, we take the corpus and iterate through it and get the co-occurrence of each word with other words in the corpus.
- We get a co-occurrence matrix through this. The words which occur next to each other get a value of 1, if they are one word apart then $1/2$, if two words apart then $1/3$ and so on.
- Let us take an example to understand how the matrix is created. We have a small corpus:

Corpus:

It is a nice evening.

Good Evening!

Is it a nice evening?

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>



	it	is	a	nice	evening	good
it	0					
is	1+1	0				
a	1/2+1	1+1/2	0			
Nice	1/3+1/2	1/2+1/3	1+1	0		
Evening	1/4+1/3	1/3+1/4	1/2+1/2	1+1	0	
good	0	0	0	0	1	0